Project - High Level Design

on

Chat with Energy Doc. (RAG)

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1.	KHUSHI KUDADE	EN22CS304034
2.	SARTHAK KASAT	EN22CS306048
3.	RISHABH GUPTA	EN22CS304054
4.	SAKSHI TIWARI	EN22CS304055
5.	MOHAMMAND ZAID KHAN	EN22EE301011

Group Name: 07D11

Project Number: GAI-33

Industry Mentor Name:

University Mentor Name: HEMLATA PATEL

Academic Year:2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1.Introduction

The project titled *"Chat with Energy Documents (RAG)"* is designed to develop an intelligent system that allows users to interact with domain-specific documents using natural language queries. The system is based on the Retrieval-Augmented Generation (RAG) approach, which combines information retrieval techniques with large language models to generate accurate and context-aware responses.

In this system, users can upload energy-related documents such as PDF reports, technical manuals, and research papers. The system processes these documents by extracting text, dividing it into smaller chunks, and converting them into vector embeddings. These embeddings are stored in a vector database for efficient retrieval.

When a user asks a question, the system retrieves the most relevant document chunks using similarity search and passes them to a Large Language Model (LLM). The LLM then generates a response strictly based on the retrieved content, ensuring that the answer is grounded in the provided documents.

The main objective of this project is to build a reliable and efficient question-answering system that minimizes incorrect or hallucinated responses and provides accurate information from domain-specific sources.

2. Scope of the Document

This High-Level Design (HLD) document defines the overall architecture and design of the "Chat with Energy Documents (RAG)" system. It provides a structured overview of how the system is organized and how different components interact with each other.

The scope of this document includes system architecture, application design, component design, data handling mechanisms, and API interactions. It also covers key aspects such as security, performance, and scalability considerations required for building an efficient and reliable system.

This document focuses only on high-level design and does not include detailed implementation or coding-level specifications.

The scope of this document includes system architecture, application design, component design, data handling mechanisms, and API interactions. It also covers key aspects such as security, performance, and scalability considerations required for building an efficient and reliable system.

3. Intended Audience

This document is intended for various stakeholders involved in the development and evaluation of the system.

It provides a clear understanding of the system architecture, design decisions, and overall workflow so that different users can interpret the system easily.

The primary audience includes:

- **Developers:**
  To understand system structure, modules, and implementation flow.
- **Project Mentors and Evaluators:**
  To review the design approach and assess the project quality.
- **System Architects:**
  To analyze architectural decisions and scalability aspects.
- **Students/Team Members:**
  To get a clear understanding of how different components of the system interact.

This document ensures that all stakeholders have a common understanding of the system at a high level.

4. System Overview

The "Chat with Energy Documents (RAG)" system is designed to enable users to interact with energy-related documents using a conversational interface. The system leverages Retrieval-Augmented Generation (RAG) to ensure that responses are accurate and strictly based on the uploaded documents.

The system works by processing documents, converting them into embeddings, and storing them in a vector database. When a user submits a query, the system retrieves the most relevant information and generates a response using a Large Language Model (LLM).

The system provides the following key functionalities:

- **Document Upload:**
  Users can upload energy-related documents such as PDFs, manuals, and reports.
- **Text Processing:**
  The system extracts and processes text from uploaded documents.

- **Embedding Generation:**
  Text is converted into vector embeddings for efficient search.
- **Vector Storage:**
  Embeddings are stored in a vector database for fast retrieval.
- **Query Handling:**
  Users can ask questions in natural language.
- **Information Retrieval:**
  Relevant document chunks are retrieved using similarity search.
- **Response Generation:**
  The LLM generates accurate answers based on retrieved data.

Key Characteristics of the System:

- Provides domain-specific answers
- Reduces incorrect or hallucinated responses
- Ensures context-aware interaction
- Supports scalable document processing

This system is particularly useful for analyzing large volumes of technical energy documents efficiently.

5. System Design

The system follows a modular and layered architecture to ensure scalability, maintainability, and efficient processing of data. It is divided into three major layers: Frontend, Backend, and Data Layer.

The frontend handles user interaction, the backend manages processing and communication, and the data layer is responsible for storing embeddings and document-related data.

The system is designed based on the following layers:

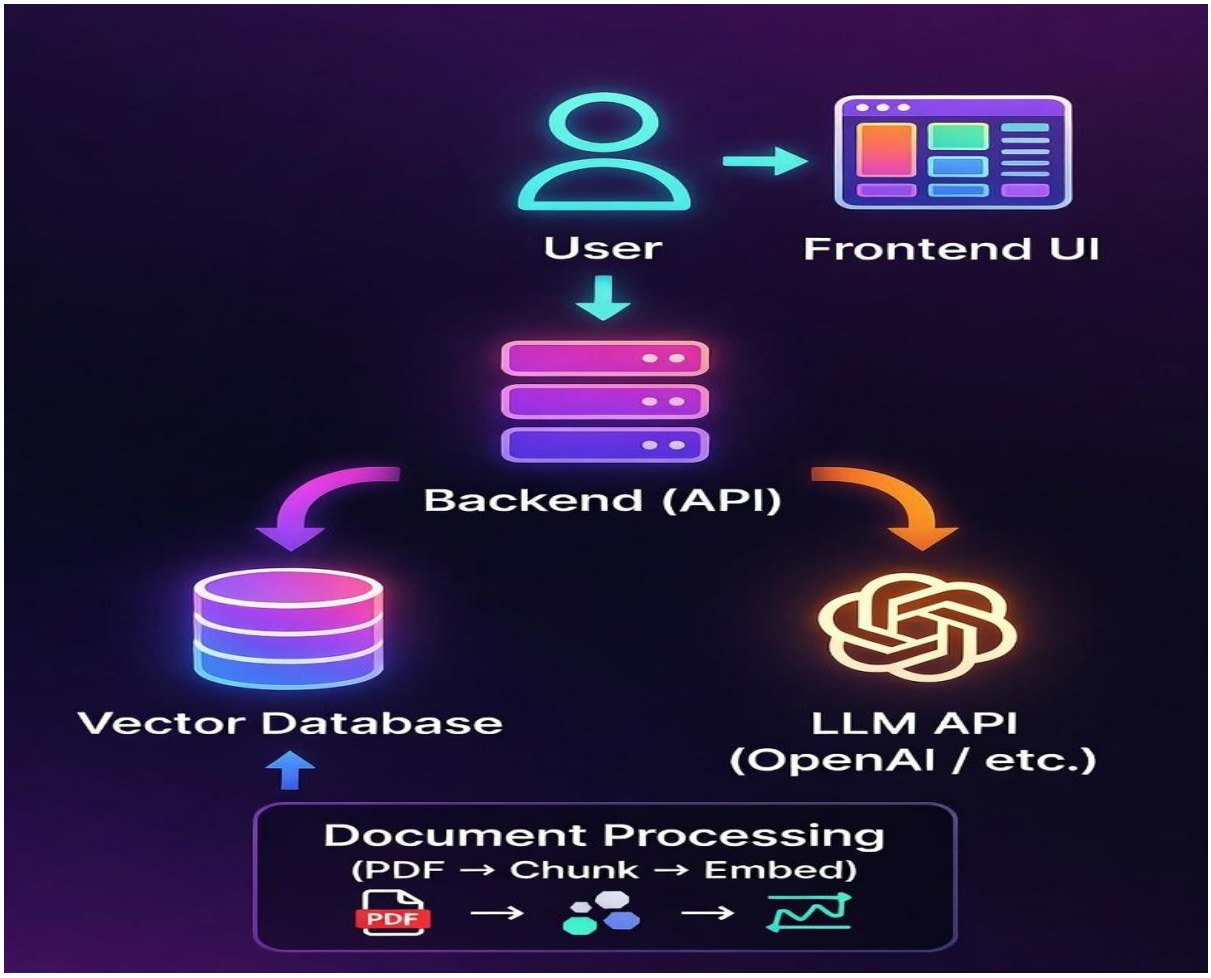
- **Frontend Layer:**
  Provides user interface for uploading documents and asking queries. It displays the generated responses to the user.
- **Backend Layer:**
  Acts as the core processing unit. It handles document processing, embedding

- generation, API management, and communication with external services like vector databases and LLM APIs.
- **Data Layer:**
  Stores document embeddings and metadata using a vector database, enabling fast similarity-based retrieval.

Key Design Features:

- Modular architecture for easy maintenance
- Separation of concerns between layers
- Efficient data processing using embeddings
- Integration with external APIs (LLM)
- Scalable design for handling large datasets

FIG:1-Architecture Diagram:



6. Application Design

The application is designed using a modular approach, where each module is responsible for a specific functionality. This separation helps in improving maintainability, scalability, and ease of development.

The system is mainly divided into three major modules: Frontend, Backend, and Database.

The main modules of the application are:

- **Frontend Module:**
  This module provides the user interface through which users interact with the system. It allows users to upload documents and ask questions. The responses generated by the system are displayed in a readable format.
- **Backend Module:**
  The backend acts as the core processing unit of the system. It handles user requests, processes documents, generates embeddings, and communicates with the vector database and LLM APIs.
- **Database Module:**
  This module is responsible for storing document data, embeddings, and metadata. It ensures fast retrieval of relevant information using vector similarity search.

Functional Responsibilities of Each Module:

- Frontend handles user interaction and display
- Backend manages logic, processing, and API communication
- Database ensures efficient storage and retrieval

Advantages of this Design:

- Easy to maintain and update individual modules
- Supports scalability for large datasets
- Improves system performance through separation of concerns
- Enables integration with different technologies

7. Process Flow

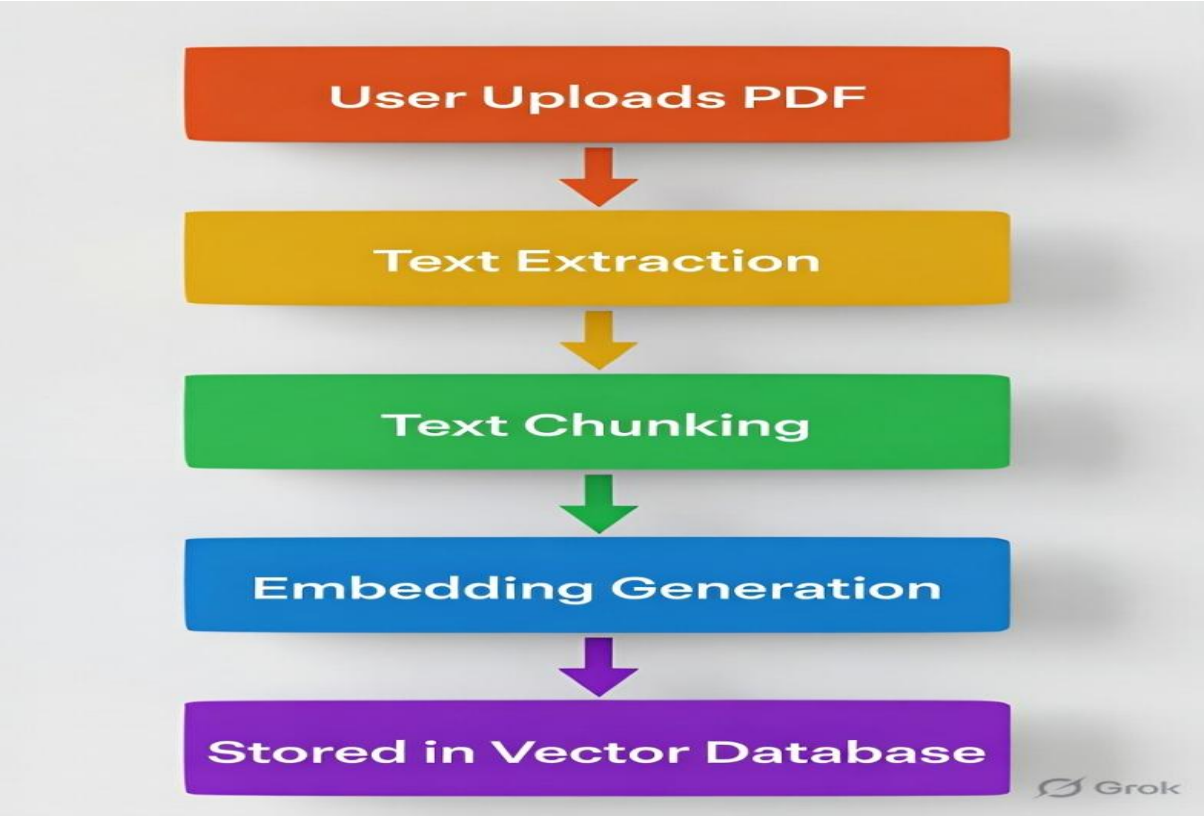
The system follows two major workflows: **Document Processing Flow** and **Query Processing Flow**. These flows describe how data moves through the system from input to output.

7.1 Document Processing Flow

This process starts when a user uploads a document into the system. The system processes the document step-by-step to make it ready for querying.

Steps involved:

- User uploads a PDF document
- The system extracts text from the document
- Extracted text is divided into smaller chunks
- Each chunk is converted into vector embeddings
- Generated embeddings are stored in the vector database



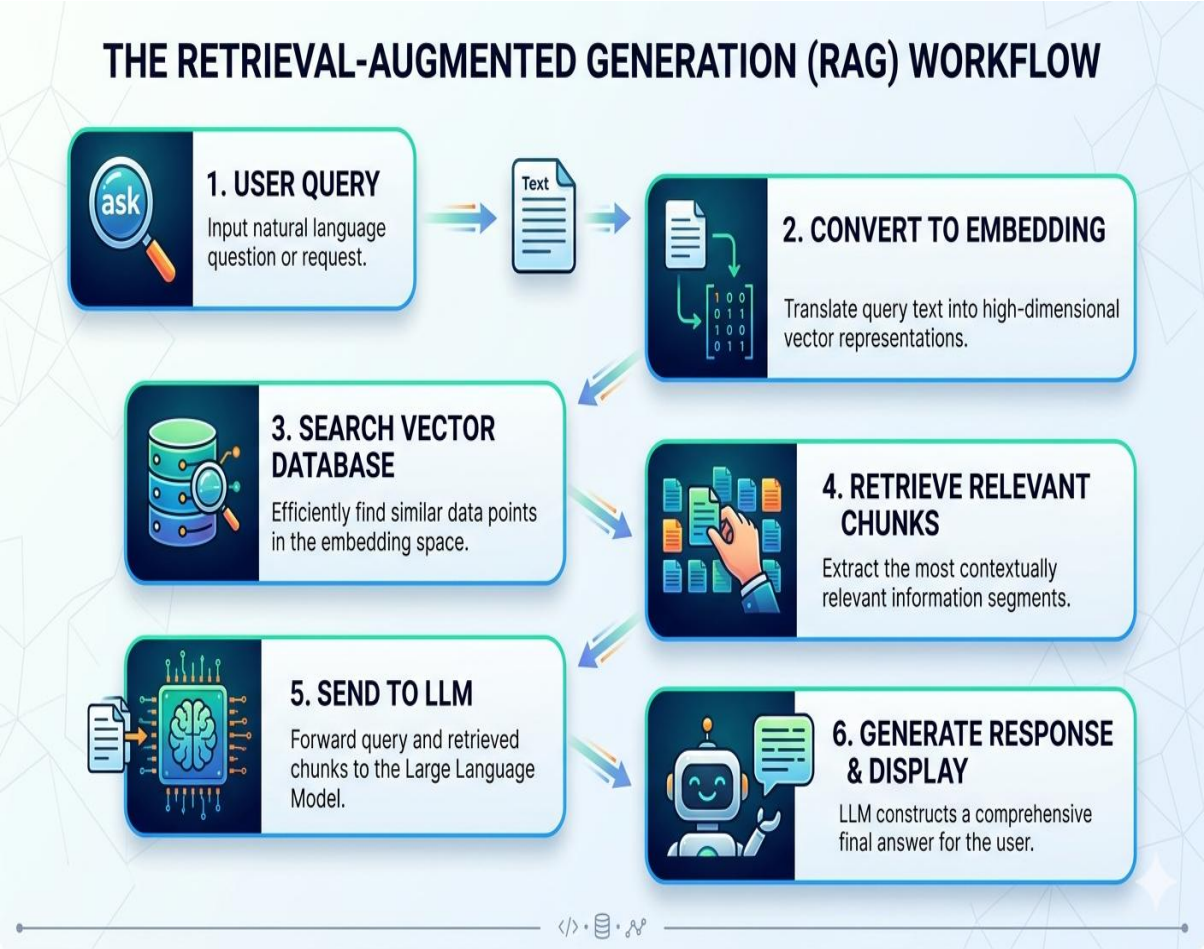
7.2 Query Processing Flow

This process starts when a user asks a question. The system retrieves relevant information and generates a response.

Steps involved:

- User enters a query
- Query is converted into embedding
- System searches the vector database
- Relevant document chunks are retrieved
- Retrieved data is sent to the LLM
- LLM generates a response
- Response is displayed to the user

This ensures that answers are accurate and based only on the document content.\



8. Information Flow

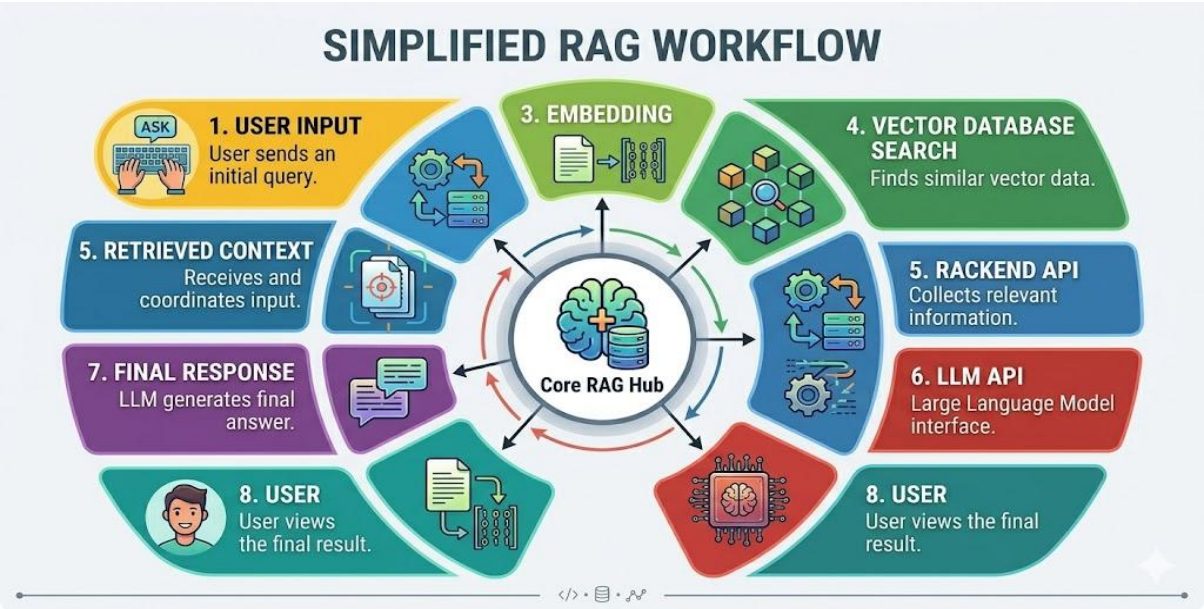
The information flow describes how data moves within the system during both document processing and query handling. It ensures a clear understanding of how input is transformed into output through different components.

When a user interacts with the system, the request is first received by the backend API. The backend processes the input and forwards it to the embedding model to convert it into vector format. These embeddings are then used to search the vector database for relevant information.

The retrieved data is passed to the Large Language Model (LLM), which generates a context-aware response. Finally, the generated response is sent back to the frontend and displayed to the user.

Flow of Information in the System:

- User provides input (document or query)
- Backend API processes the input
- Embedding model converts input into vectors
- Vector database retrieves relevant data
- Retrieved data is sent to LLM
- LLM generates response
- Response is returned to the user



9. Components Design

The system is composed of multiple components, each responsible for a specific functionality. These components work together to enable efficient document processing and accurate response generation.

Each component is designed in a modular way to ensure flexibility, scalability, and ease of maintenance.

Main Components of the System

1. PDF Loader
This component is responsible for extracting text from uploaded PDF documents. It ensures that the content is converted into a readable format for further processing.

2. Text Chunker
After text extraction, this component divides large text into smaller chunks. This improves retrieval accuracy and ensures efficient embedding generation.

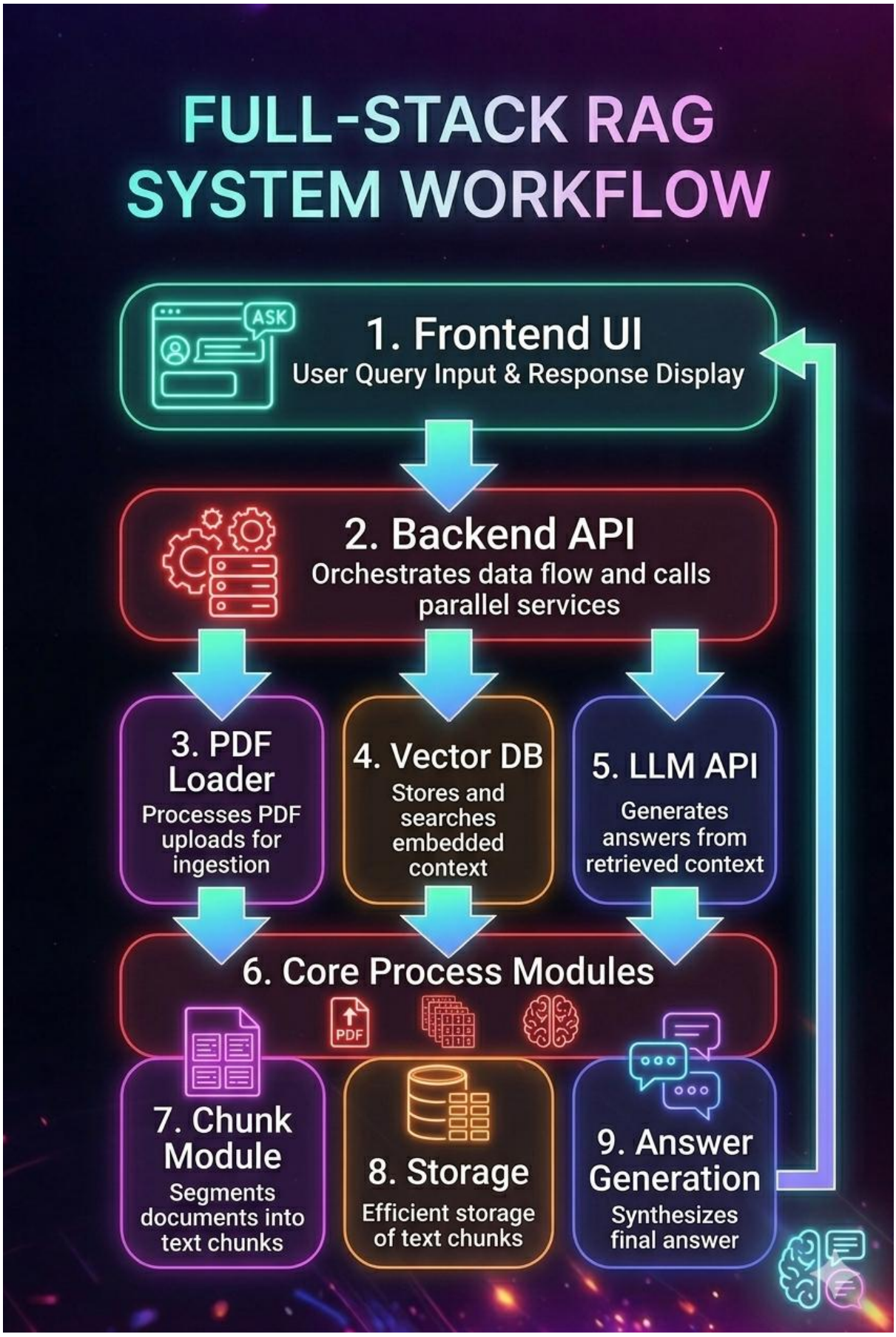
3. Embedding Generator
This component converts text chunks into vector embeddings using pre-trained models. These embeddings represent the semantic meaning of the text.

4. Vector Database
The vector database stores embeddings along with metadata. It enables fast similarity-based search to retrieve relevant information.

5. Retriever
This component searches the vector database using the query embedding and retrieves the most relevant document chunks.

6. LLM Module
The Large Language Model processes the retrieved chunks and generates a meaningful and context-aware response.

7. Backend API
Acts as the central controller that connects all components. It handles requests, manages workflows, and communicates with external services.


FULL-STACK RAG SYSTEM WORKFLOW
ASK
1. Frontend UI
User Query Input & Response Display
2. Backend API
Orchestrates data flow and calls parallel services
3. PDF Loader
Processes PDF uploads for ingestion
4. Vector DB
Stores and searches embedded context
5. LLM API
Generates answers from retrieved context
6. Core Process Modules
PDF
7. Chunk Module
Segments documents into text chunks
8. Storage
Efficient storage of text chunks
9. Answer Generation
Synthesizes final answer

10. Key Design Considerations

While designing the "Chat with Energy Documents (RAG)" system, several important factors were considered to ensure efficiency, accuracy, and scalability. These considerations help in building a robust and reliable system.

Major Design Considerations

- **Accuracy of Retrieval:**
  The system must retrieve the most relevant document chunks to ensure correct and meaningful responses. Poor retrieval can lead to incorrect answers.
- **Chunk Size Optimization:**
  Choosing an appropriate chunk size is important. Smaller chunks improve precision, while larger chunks provide more context. A balance is required.
- **Embedding Quality:**
  High-quality embeddings are essential for better similarity search and accurate information retrieval.
- **System Scalability:**
  The system should handle large volumes of documents without performance degradation. The design supports scaling using efficient databases and modular architecture.
- **Response Latency:**
  The system should generate responses quickly. Efficient retrieval and optimized API calls help reduce delay.
- **Data Security:**
  Uploaded documents may contain sensitive information. Proper security measures must be implemented to protect data.
- **Integration with External APIs:**
  The system depends on LLM APIs, so proper handling of API calls, rate limits, and failures is important.
- **Maintainability:**
  The modular design ensures that components can be updated or replaced without affecting the entire system.

11. API Catalogue

The system exposes a set of APIs to handle document processing and user queries. These APIs act as an interface between the frontend and backend, enabling smooth communication and functionality.

Each API is designed to perform a specific task within the system workflow.

List of APIs

1. Upload API

- **Endpoint:** /upload
- **Method:** POST
- **Description:**
  This API is used to upload PDF documents into the system. The uploaded file is stored temporarily for processing.

---

2. Process API

- **Endpoint:** /process
- **Method:** POST
- **Description:**
  This API processes the uploaded document by extracting text, splitting it into chunks, generating embeddings, and storing them in the vector database.

---

3. Query API

- **Endpoint:** /query
- **Method:** POST
- **Description:**
  This API accepts user queries in natural language. It converts the query into embeddings and retrieves relevant document chunks from the vector database.

---

4. Response API

- **Method:** GET
- **Description:**
  This API returns the final response generated by the LLM after processing the query and retrieved data.

12. Data Design

The data design defines how data is collected, processed, stored, and utilized within the "Chat with Energy Documents (RAG)" system. Since the system primarily deals with unstructured data in the form of documents, an efficient data design is essential for accurate retrieval and response generation.

The system processes documents by extracting text, converting it into structured chunks, and generating embeddings that are stored for similarity-based search.

Types of Data Used in the System:

- **Raw Documents:**
  These include PDF files such as energy reports, manuals, and technical documents uploaded by users.
- **Extracted Text:**
  Text content extracted from documents during preprocessing.
- **Text Chunks:**
  Large text is divided into smaller chunks to improve processing efficiency and retrieval accuracy.
- **Embeddings:**
  Numerical vector representations of text chunks that capture semantic meaning.
- **Metadata:**
  Includes document ID, chunk ID, file name, and other identifiers used for tracking and organization.

Data Storage Strategy:

- Raw documents are stored in file storage or local system
- Embeddings and metadata are stored in a vector database
- Data is indexed for fast similarity search

13. Data Model

The data model defines the structure and relationships between different data entities in the system. It ensures organized storage and efficient retrieval of information.

The system mainly uses two entities: **Document** and **Chunk**.

Document Entity:

- Document ID
- Document Name
- Document Content

Chunk Entity:

- Chunk ID
- Document ID (Foreign Key)
- Chunk Text
- Embedding Vector

Relationship:

- One document can have multiple chunks
- Each chunk is linked to a single document

14. Data Access Mechanism

The data access mechanism defines how data is retrieved and managed within the system. The system uses vector-based similarity search instead of traditional keyword-based search.

When a query is received, it is converted into an embedding. This embedding is compared with stored embeddings in the vector database to find the most relevant matches.

Key Mechanisms:

- **Vector Similarity Search:**
  Uses techniques like cosine similarity to find similar embeddings.
-

- **Indexed Retrieval:**
  Embeddings are indexed to improve search speed.
- **API-Based Access:**
  All data interactions are handled through backend APIs.

Advantages:

- Faster retrieval compared to traditional methods
- Context-aware search results
- High accuracy in matching relevant data

---

15. Data Retention Policies

Data retention policies define how long data is stored and managed in the system. Proper data retention ensures efficient storage usage and compliance with system requirements.

Retention Strategy:

- Documents are stored until deleted by the user
- Embeddings are retained for faster future queries
- Logs are stored temporarily for monitoring and debugging

Policies Followed:

- Avoid unnecessary duplication of data
- Maintain only relevant and useful data
- Ensure data consistency and integrity

---

16. Data Migration

Data migration refers to the process of updating or transferring data when system changes occur, such as upgrading models or modifying storage systems.

In this system, migration is important when embedding models are updated or when data needs to be reprocessed.

Migration Scenarios:

- Re-generating embeddings when a new model is used

- Moving data to a new vector database
- Updating metadata structure

Approach:

- Reprocess documents when required
- Maintain backup before migration
- Ensure no data loss during transition

17. Interfaces

Interfaces define how different components of the system interact with each other and with external systems. The "Chat with Energy Documents (RAG)" system uses multiple interfaces for smooth communication.

Types of Interfaces:

- **User Interface (UI):**
  Allows users to upload documents and ask queries. It displays the generated responses in a readable format.
- **Backend API Interface:**
  Acts as a communication layer between frontend and backend services. It handles all requests and responses.
- **LLM API Interface:**
  Connects the system with external Large Language Model services for generating responses.
- **Vector Database Interface:**
  Enables storage and retrieval of embeddings using similarity search.

---

18. State and Session Management

The system follows a stateless architecture where each request is processed independently. This improves scalability and simplifies system design.

Key Points:

- No session data is stored on the server side
- Each request contains all necessary information

- Token-based mechanisms can be used for user identification
- Ensures better performance and scalability

Advantages:

- Easy to scale the system
- Reduces server load
- Improves reliability

---

19. Caching

Caching is used to improve system performance by reducing repeated computations and API calls.

Caching Strategies:

- Frequently asked queries can be cached
- Embeddings for repeated inputs can be reused
- API responses can be temporarily stored

Benefits:

- Reduces response time
- Decreases load on LLM APIs
- Improves overall system efficiency

---

20. Non-Functional Requirements

Non-functional requirements define the quality attributes of the system.

Key Requirements:

- **Scalability:**
  System should handle increasing number of documents and users.
- **Reliability:**
  System should provide consistent and correct results.
- **Availability:**
  System should be accessible whenever required.

- **Maintainability:**
  Easy to update and manage system components.
- **Usability:**
  User interface should be simple and easy to use.

---

21. Security Aspects

Security is an important aspect of the system as it deals with potentially sensitive documents.

Security Measures:

- Secure API communication (HTTPS)
- Authentication and authorization mechanisms
- Data encryption for stored information
- Protection against unauthorized access

Additional Considerations:

- Prevent data leakage
- Ensure safe API usage
- Handle user data securely

---

22. Performance Aspects

Performance defines how efficiently the system operates under different conditions.

Key Performance Factors:

- Fast document processing
- Low latency in query response
- Efficient vector search
- Optimized API calls

Optimization Techniques:

- Use of caching
- Efficient indexing in vector database

- Reducing unnecessary computations

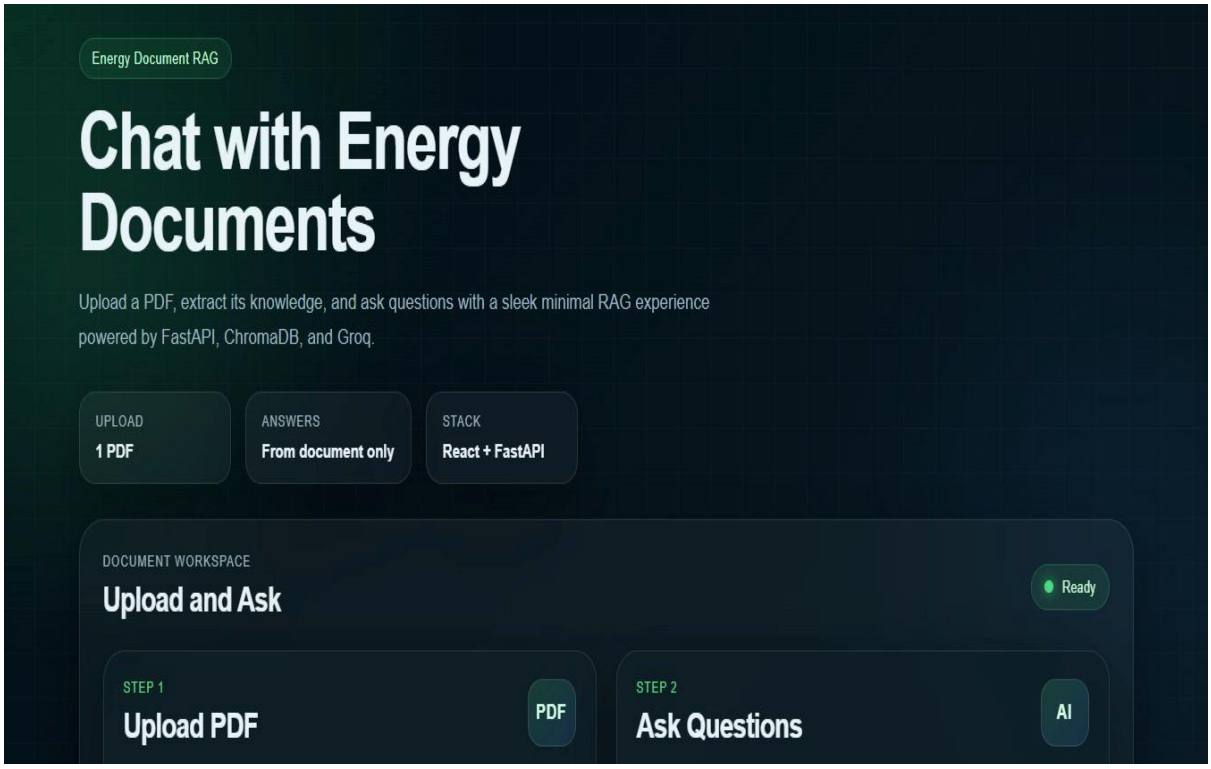
---

23. References

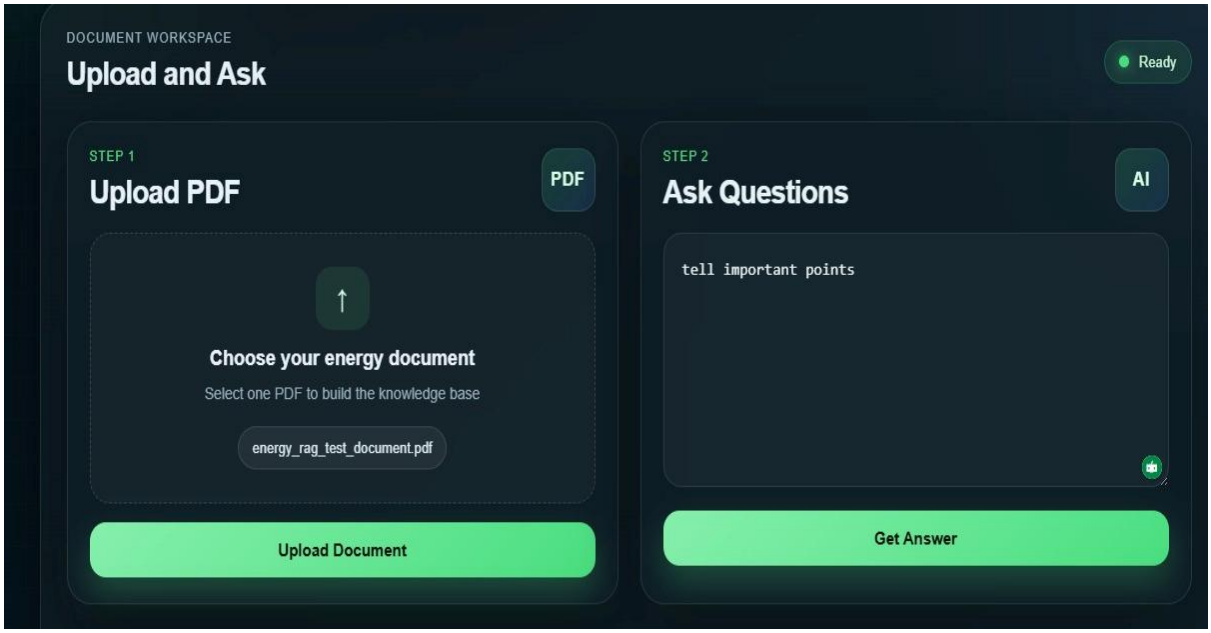
- Retrieval-Augmented Generation (RAG) concepts
- Vector Database documentation
- Large Language Model (LLM) APIs
- Python libraries such as LangChain, FAISS

Screenshot of the Project:

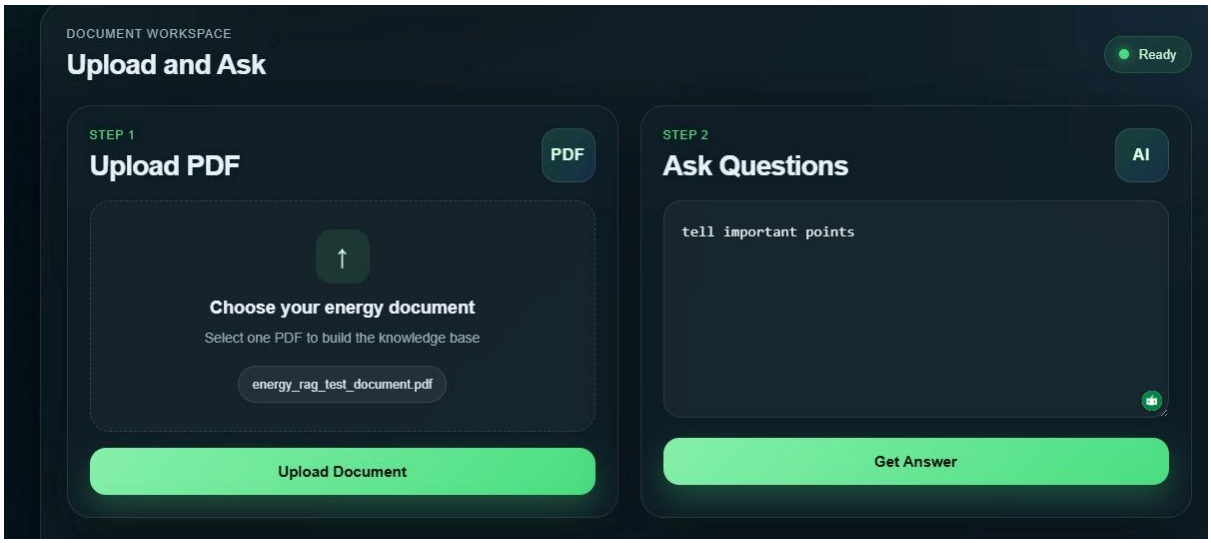
1.Landing Page->



2. Upload section->



3.Response page:



----------X----------

THANK YOU